

Kerberoasting Attack

CTC-Bank Active Directory Environment

Name: *Matyas Abraham*

IDNO: *CTC-207-26*

What I did, in order

- Ran nmap against 10.60.1.11 and found it's a Windows Server 2022 domain controller (CTCBANK-DC) with Kerberos, LDAP, and SMB open.
- Used a null SMB session and lookupsid.py to RID-cycle the domain and build a list of usernames (users.txt).
- Out of curiosity I first tried GetNPUsers.py against that user list to check for AS-REP Roastable accounts (no preauth required). One account, meseret.alemu, came back vulnerable and I cracked the AS-REP hash with hashcat (mode 18200, rockyou.txt) to get meseret.alemu:Password1. This is a different attack from Kerberoasting, but it gave me my first foothold and valid domain credentials to use for the actual Kerberoasting step.
- Password-sprayed Password1 across the rest of the user list and picked up a second valid account, bethlehem.tadesse.
- Used those creds with BloodHound to map out ACLs and found bethlehem.tadesse had rights to reset tigest.haile's password, which gave me a third working account.
- With tigest.haile's credentials, I ran GetUserSPNs.py against the domain to enumerate service accounts and request a TGS ticket - this is the actual Kerberoasting step.
- Hit two snags along the way: an LDAP invalidCredentials error (I was using a stale password after a reset - fixed by resetting it back), and a Kerberos KRB_AP_ERR_SKEW error caused by my Kali VM's clock drifting from the DC by hours (fixed with ntpdate against the DC, after stopping the background time-sync service that kept pulling it back).
- Once the clock was in sync, GetUserSPNs.py successfully returned a TGS ticket hash for the db_svc account.
- Took the ticket hash offline and tried to crack it with hashcat mode 13100, first against rockyou.txt alone, then against rockyou.txt with the best66.rule ruleset. Both runs completed (exhausted the keyspace) without recovering the password.

Findings

Item	Detail
Domain	ctcbank.corp (CTCBANK-DC, Windows Server 2022)
Kerberoastable SPN account	db_svc - MSSQLSvc/db01.ctcbank.corp:1433
Password last set	2026-08-05 17:40:47
Tool used to request TGS	Impacket GetUserSPNs.py
Ticket format obtained	\$krb5tgs\$23\$*db_svc\$CTCBANK.CORP\$ctcbank.corp/db_svc*... (etype 23 / RC4)
Cracking tool	hashcat -m 13100
Wordlists / rules tried	rockyou.txt, then rockyou.txt + best66.rule
Result	Not cracked - both attempts exhausted the full keyspace with 0/1 recovered

Assignment Questions

Q1. What is an SPN and why is it required for this attack?

An SPN (Service Principal Name) is basically the label Kerberos uses to identify which service is running under which account, on which host - for example MSSQLSvc/db01.ctcbank.corp:1433 tells the domain "the SQL service on db01 runs as this account." It's required for Kerberoasting because the whole attack is built around asking the KDC for a service ticket, and the KDC can only issue a ticket for something it can identify - that's the SPN. No SPN, nothing to request a ticket for.

Q2. Why can this attack be performed without admin privileges?

Because requesting a TGS ticket is just normal Kerberos behavior, not a privileged action. Any authenticated domain user is allowed to ask the KDC for a service ticket to any SPN in the domain - that's how Kerberos is supposed to work, so a regular user can talk to a file share or a SQL server without an admin getting involved every time. The catch is that the ticket comes back encrypted with the service account's own password hash, and nothing stops me from taking that encrypted blob home and trying to crack it offline. I proved this myself - I used tigist.haile, a completely ordinary domain user with no admin rights, and it worked fine.

Q3. What's the difference between Kerberoasting and AS-REP Roasting?

Kerberoasting targets the TGS step - I ask for a service ticket for an account that has an SPN (usually a service account like db_svc), and I need valid domain credentials to do it. AS-REP Roasting targets the earlier AS step - it goes after accounts that have "Do not require Kerberos preauthentication" enabled, and I don't need any credentials at all to try it, since preauth is the part that's normally supposed to prove I know the password before the KDC replies. I actually ran into both in this lab: GetNPUsers.py against meseret.alemu was AS-REP Roasting (no creds needed, and it's what got me in the door), and GetUserSPNs.py against db_svc using tigist.haile's account was the real Kerberoasting.

Q4. What's the difference between Kerberoasting and AS-REP Roasting? (repeat)

Same answer as Q3 - to put it in one line: AS-REP Roasting abuses a misconfiguration (preauth disabled) and needs no credentials; Kerberoasting abuses normal, expected Kerberos behavior (any user can request any service ticket) and needs valid credentials. Both end with an offline-crackable hash, just from different points in the Kerberos flow.

Q5. List the SPN accounts you found in the CTC-Bank domain.

Running GetUserSPNs.py against the domain returned one SPN-registered account:

- db_svc - MSSQLSvc/db01.ctcbank.corp:1433 (password last set 2026-08-05, never logged on interactively)

Q6. Include a screenshot showing you successfully requested a TGS ticket in the write-up.

See attached screenshot - GetUserSPNs.py ctcbank.corp/tigist.haile:Password1 -dc-ip 10.60.1.11 -request, showing the db_svc SPN and the full \$krb5tgs\$23\$db_svc\$CTCBANK.CORP\$... ticket hash returned after I fixed the clock-skew issue.

Q7. Which tool(s) did you use to request/extract the ticket?

Impacket's GetUserSPNs.py for enumerating SPNs and requesting the ticket. I also used netexec and ntpdate along the way to fix authentication and clock-sync problems that were blocking the request.

Q8. Did you attempt to crack the ticket offline? (yes/no)

Yes.

Q9. Include proof of the cracking attempt in your write-up.

See attached screenshots - two hashcat runs against the db_svc TGS hash, mode 13100:

- hashcat -m 13100 tgs_hash.txt rockyou.txt - exhausted, 0/1 recovered
- hashcat -m 13100 tgs_hash.txt rockyou.txt -r best66.rule - exhausted at 100% (946,729,410 candidates), 0/1 recovered

db_svc's password held up against both attempts, so I didn't get a plaintext password for it - but the attack itself, from SPN enumeration through to a completed offline cracking attempt, worked end to end.

Q10. What's one mitigation you'd recommend to CTC-Bank to prevent Kerberoasting?

Move service accounts like db_svc to gMSAs (Group Managed Service Accounts). A gMSA gets a 120+ character random password that's rotated automatically and never typed or known by a human, so even if someone Kerberoasts it, the ticket is effectively uncrackable offline. Where gMSAs aren't an option, the next best thing is a long, truly random password on the service account (25+ characters) and disabling RC4 so tickets are only issued using AES, which raises the cracking cost a lot. Honestly, db_svc not cracking against rockyou + rules in this lab is a decent real-world example of why a strong service account password matters.

Q11. How long did the challenge take you?

Around 3 hours of hands-on time total, spread across a few sessions over the day. A chunk of that went into troubleshooting rather than the attack itself - a stale password causing an LDAP bind failure, and a Kerberos clock-skew error from my Kali VM's clock drifting (turned out something kept resyncing it backward, so I had to stop the background time-sync service before ntpdate would actually stick).